

这里各步迭代中的操作次序,与图9.4(a)和(b)基本相反。略微不同之处在于,因必然是整塔删除,故可省略纵向链接的调整。

其中`clear()`接口用以删除表中所有节点,在代码9.3中也是析构过程中的主要操作。

■ 时间复杂度

如代码9.11所示,词条删除算法所需的时间,不外乎消耗于两个方面。

首先是查找目标关键码,由9.2.6节的结论可知,这部分时间的期望值不过 $O(\log n)$ 。其次是拆除与目标关键码相对应的塔,这是一个自顶而下逐层迭代的过程,故累计不超过 h 步;另外,由代码9.12可见,各层对应节点的删除仅需常数时间。

综合以上分析可知,跳转表词条删除操作所需的时间不超过 $O(h) = O(\log n)$ 。

§ 9.3 散列表

散列作为一种思想既朴素亦深刻,作为一种技术则虽古老却亦不失生命力,因而在数据结构及算法中占据独特而重要地位。此类方法以最基本的向量作为底层支撑结构,通过适当的散列函数在词条的关键码与向量单元的秩之间建立起映射关系。理论分析和实验统计均表明,只要散列表、散列函数以及冲突排解策略设计得当,散列技术可在期望的常数时间内实现词典的所有接口操作。也就是说,就平均时间复杂度的意义而言,可以使这些操作所需的运行时间与词典的规模基本无关。尤为重要的是,散列技术完全摒弃了“关键码有序”的先决条件,故就实现词典结构而言,散列所特有的通用性和灵活性是其它方式无法比拟的。

以下将围绕散列表、散列函数以及冲突排解三个主题,逐层深入地展开介绍。

9.3.1 完美散列

■ 散列表

散列表(`hashtable`)是散列方法的底层基础,逻辑上由一系列可存放词条(或其引用)的单元组成,故这些单元也称作桶(`bucket`)或桶单元;与之对应地,各桶单元也应按其逻辑次序在物理上连续排列。因此,这种线性的底层结构用向量来实现再自然不过。为简化实现并进一步提高效率,往往直接使用数组,此时的散列表亦称作桶数组(`bucket array`)。若桶数组的容量为 R ,则其中合法秩的区间 $[0, R)$ 也称作地址空间(`address space`)。

■ 散列函数

一组词条在散列表内部的具体分布,取决于所谓的散列(`hashing`)方案——事先在词条与桶地址之间约定的某种映射关系,可描述为从关键码空间到桶数组地址空间的函数:

`hash() : key → hash(key)`

这里的`hash()`称作散列函数(`hash function`)。反过来,`hash(key)`也称作`key`的散列地址(`hashing address`),亦即与关键码`key`相对应的桶在散列表中的秩。

■ 实例

以学籍库为例。若某高校2011级共计4000名学生的学号为2011-0000至2011-3999,则可直接使用一个长度为4000的散列表`A[0~3999]`,并取

`hash(key) = key - 20110000`

从而将学号为`x`的学生学籍词条存放于桶单元`A[hash(x)]`。

如此散列之后, 根据任一合法学号, 都可在 $O(1)$ 时间内确定其散列地址, 并完成一次查找、插入或删除。空间性能方面, 每个桶恰好存放一个学生的学籍词条, 既无空余亦无重复。这种在时间和空间性能方面均达到最优的散列, 也称作完美散列(perfect hashing)。

实际上, Bitmap结构(习题[2-34])也可理解为完美散列的一个实例。其中, 为每个可能出现的非负整数, 各分配了一个比特位, 作为判定它是否属于当前集合的依据; 散列函数也再简单不过——各比特位在内部向量中的秩, 就是其所对应整数的数值。

遗憾的是, 以上实例都是在十分特定的条件下才成立的, 完美散列实际上并不常见。而在更多的应用环境中, 为兼顾空间和时间效率, 无论散列表或散列函数都需要经过更为精心的设计。以下就是一个更具一般性的实例。

9.3.2 装填因子与空间利用率

■ 电话查询系统

假设某大学拟建立一个电话簿查询系统, 覆盖教职员和学生所使用的共约25000门固定电话。以下, 主要考查其中反查功能的实现, 即如何高效地由电话号码获取机主的信息。

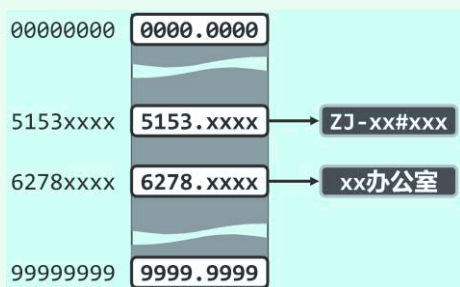


图9.5 直接使用线性数组实现电话簿词典

这一任务从数据结构的角度可理解为, 设计并实现一个词典结构, 以电话号码为词条关键码, 支持根据这种关键码的高效查询。若考虑到开机、撤机和转机等情况, 还应支持词条的插入和删除等动态操作。仿照学籍库的例子, 可如图9.5引入向量, 将电话号码为 x 的词条存放在秩为 x 的单元。如此, 不仅词条与桶单元一一对应, 而且无论是静态的查找还是动态的插入和删除, 每次操作仅需常数时间!

然而进一步分析之后不难发现, 这一方案在此情况下并不现实。从理论上讲, 在使用8位编号系统时, 整个城市固定电话最多可能达到 10^8 门。尽管该校人员所涉及的固定电话仅有25000门, 但号码却可能随机分布在[0000-0000, 9999-9999]的整个范围内。这就意味着, 上述方案所使用数组的长度大致应与 10^8 相当。每个词条占用的空间即便按100字节估计, 该数组也至少需要占用10GB的空间。也就是说, 此时的空间有效利用率仅为 $25000 / 10^8 = 0.025\%$, 绝大部分的空间实际上处于闲置状态。

■ IP节点查询

另一个类似的例子是, 根据IP地址获取对应的域名信息。按照32bit地址的协议, 理论上可能的IP地址共有 $2^{32} = 4 \times 10^9$ 个, 故此时若直接套用以上方法采用最简单的散列表和散列函数, 将动辄征用100~1000GB的空间。另一方面, 尽管大多数IP并没有指定域名, 但任一IP都有可能具有域名, 故这种方法的空间利用率也仅为5%左右^⑤。而在未来采用IPv6协议之后, 尽管实际运

^⑤ 据威瑞信(VeriSign)公司2010年11月发布的《2010年第三季度域名行业报告》, 截至2010年第三季度底, 全球顶级域名(Top Level Domain, TLD)的注册总数已达到2.02亿, 平均约每20个IP中才有一个IP具有域名

行中的节点数目在短时间内不会有很大的变化，但允许使用的IP地址将多达 $2^{128} = 256 \times 10^{36}$ 个——如此庞大的地址空间根本无法直接使用数组表示和存放^①；即便有如此规模的存储介质，其空间利用率依然极低。

■ 兼顾空间利用率与速度

此类问题在实际应用中十分常见，其共同的特点可归纳为：尽管词典中实际需要保存的词条数 N （比如25000门）远远少于可能出现的词条数 R （ 10^8 门），但 R 个词条中的任何一个都有可能出现在词典中。仿照2.4.1节针对向量空间利用率的度量方法，这里也可以将散列表中非空桶的数目与桶单元总数的比值称作装填因子（load factor）。从这一角度来看，上述问题的实质在于散列表的装填因子太小，从而导致空间利用率过低。

无论如何，散列方法的查找和更新速度实在诱人，也的确可以完美地适用于学籍库之类的应用。那么，能否在保持优势的前提下，克服其在存储空间利用率方面的不足呢？答案是肯定的，但需要运用一系列的技巧，其中首先就是散列函数的设计。

9.3.3 散列函数

9.3.10节将介绍一般类型关键码到整数的转换方法，故不妨先假定关键码均为 $[0, R)$ 范围内的整数。将词典中的词条数记作 N ，散列表长度记作 M ，于是通常有：

$$R \gg M > N$$

如图9.6所示，散列函数 $\text{hash}()$ 的作用可理解为，将关键码空间 $[0, R)$ 压缩为散列地址空间 $[0, M)$ 。

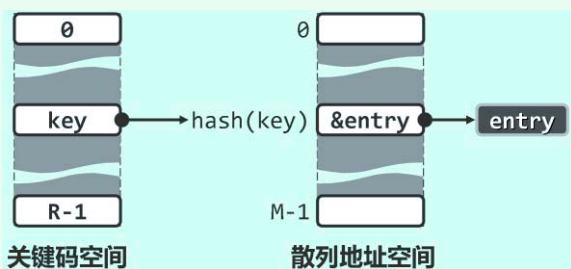


图9.6 散列函数

■ 设计原则

作为好的散列函数， $\text{hash}()$ 应具备哪些条件呢？首先，必须具有确定性。无论所含的数据项如何，词条 E 在散列表中的映射地址 $\text{hash}(E.\text{key})$ 必须完全取决于其关键码 $E.\text{key}$ 。其次，映射过程自身不能过于复杂，唯此方能保证散列地址的计算可快速完成，从而保证查询或修改操作整体的 $O(1)$ 期望执行时间。再次，所有关键码经映射后应尽量覆盖整个地址空间 $[0, M)$ ，唯此方可充分利用有限的散列表空间。也就是说，函数 $\text{hash}()$ 最好是满射。

当然，因定义域规模 R 远远大于取值域规模 M ， $\text{hash}()$ 不可能是单射。这就意味着，关键码不同的词条被映射到同一散列地址的情况——称作散列冲突（collision）——难以彻底避免。尽管9.3.5节将会介绍解决冲突的办法，但若能在设计和选择散列函数阶段提前做些细致而充分的考量，便能尽可能地降低冲突发生的概率。

在此，最为重要的一条原则就是，关键码映射到各桶的概率应尽量接近于 $1/M$ ——若关键码均匀且独立地随机分布，这也是任意一对关键码相互冲突的概率。就整体而言，这等效于将关键码空间“均匀地”映射到散列地址空间，从而避免导致极端低效的情况——比如，因大部分关键

^① 截至2010年，人类拥有的数字化数据总量为1.2ZB（1ZB = 2^{70} = 10^{21} 字节），预计到2020年可达35ZB

码集中分布于某一区间，而加剧散列冲突；或者反过来，因某一区间仅映射有少量的关键码，而导致空间利用率低下。

总而言之，随机越强、规律性越弱的散列函数越好。当然，完全符合上述条件的散列函数并不存在，我们只能通过先验地消除可能导致关键码分布不均匀的因素，最大限度地模拟理想的随机函数，尽最大可能降低发生冲突的概率。

■ 除余法 (division method)

符合上述要求的一种最简单的映射办法，就是将散列表长度 M 取作为素数，并将关键码 key 映射至 key 关于 M 整除的余数：

$$\text{hash}(key) = key \bmod M$$

仍以校园电话簿为例，若取 $M = 90001$ ，则以下关键码：

{ 6278-5001, 5153-1876, 6277-0211 }

将如图9.7所示分别映射至

{ 54304, 51304, 39514 }

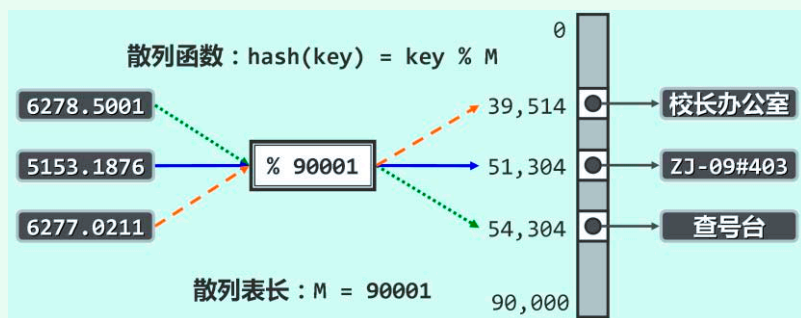


图9.7 除余法

请注意，采用除余法时必须将 M 选作素数，否则关键码被映射至 $[0, M)$ 范围内的均匀度将大幅降低，发生冲突的概率将随 M 所含素因子的增多而迅速加大。

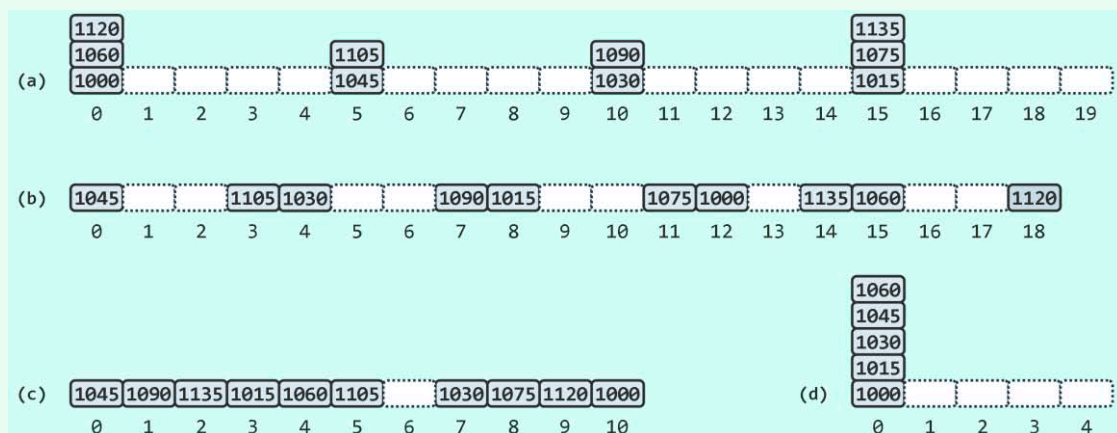


图9.8 素数表长可降低冲突的概率并提高空间的利用率